

Ejercicio 12. Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un número entero positivo. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, a lo sumo $\lfloor \sqrt{n} \rfloor$ valores están fuera del rango $[20, 40]$.

Proponer un algoritmo $O(n)$ que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

Ejercicio 18. Dado un arreglo de pares $\langle \text{estudiante}, \text{nota} \rangle$, queremos devolver un arreglo de pares $\langle \text{estudiante}, \text{promedio} \rangle$, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta.

Los estudiantes se representan con un número de libreta (puede considerar que es un número de a lo sumo 10 dígitos), y la nota es un número entre 1 y 100. La cantidad de estudiantes se considera no acotada.

Se pide dar un algoritmo que resuelva el problema con complejidad $O(n + m \log m)$, donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes.

Ejemplo:

Entrada: $\langle 12395, 72 \rangle, \langle 45615, 81 \rangle, \langle 12395, 94 \rangle, \langle 45615, 100 \rangle, \langle 78920, 81 \rangle,$
 $\langle 12395, 90 \rangle, \langle 45615, 71 \rangle, \langle 78920, 50 \rangle$

Salida: $\langle 12395, 85.3333 \rangle, \langle 45615, 84 \rangle, \langle 78920, 65.5 \rangle$

- Describa cada etapa del algoritmo con sus palabras, y justifique por qué cumple con las complejidades pedidas.
- Escriba el algoritmo en pseudocódigo. Puede utilizar (sin reescribir) todos los algoritmos y estructuras de datos vistos en clase.

```

----- EJERCICIO 12 -----
proc OrdenarMediciones(in array<int> A): array<int>{
  var enRango: listaEnlazada<int>      --0(1)
  var fueraDeRango: listaEnlazada<int> --0(1)
  var res: array<int>                  --0(1)
  for i = 0; i < |A|; i++              --0(n)
    if A[i] >= 20 && A[i] <= 40 then
      enRango.agregarAtras(A[i])      --0(1)
    else
      fueraDeRango.agregarAtras(A[i]) --0(1)
  endfor

  listToArray(fueraDeRango)           --0(n)
  listToArray(enRango)                 --0(n)
  selectionSort(fueraDeRango)         --0(√n * √n) = 0(√n²) = 0(n)
  bucketSort(enRango)                 --0(n)
  res = merge(fueraDeRango, enRango)  --0(n)
  return res
}
-- OBS: Se puede usar cualquier algoritmo de ordenamiento para "fueraDeRango".
-- COMPLEJIDAD TOTAL: 0(n) + 2*0(n) + 0(n) + 0(n) = 0(n)
--                      (for) (listToArray) (selection) (merge)

----- EJERCICIO 18 -----
proc OrdenarXPromedio(in array<(int,int)> A): array<(int,float)>{
  var resL: lista<(int,float)> := listaVacía()
  var res: array<(int,float)> := []
  var B: array<(int,float)>

  B := RadixSort(A)  -- Ordena por libreta (1er componente)
                    -- Las libretas son acotadas a 10 dígitos
                    -- Complejidad: 0(n)

  if |B| > 0
    var libretaActual: int := B[0][0]      --0(1)
    var notasAcumuladas: int := B[0][1]    --0(1)
    var actual: int := 1                   --0(1)
    var cantNotas: int := 1                --0(1)
    var promedio: float := 0.0             --0(1)

    while actual < |B|                      --0(n)
      if libretaActual == B[actual][0]      --0(1)
        notasAcumuladas := notasAcumuladas + B[actual][1] --0(1)
        cantNotas := cantNotas + 1          --0(1)
      else
        promedio := notasAcumuladas/cantNotas --0(1)
        resL.agregarAtras(tupla(libretaActual, promedio)) --0(1)

        libretaActual := B[actual][0]      --0(1)
        notasAcumuladas := B[actual][1]    --0(1)
        cantNotas := 1                     --0(1)
      endif
      actual := actual + 1                  --0(1)
    endwhile
    -- Agregamos el último estudiante!
    promedio := notasAcumuladas/cantNotas  --0(1)
    resL.agregarAtras(tupla(libretaActual, promedio)) --0(1)

    res = listToArray(resL)                --0(m)
    mergeSort(res)  -- Ordena de forma DESC por promedio (2da componente)
                  -- Desempata por 1er componente
                  -- Complejidad: 0(m*log(m))

  endif
  return res
}

```

```

}
-- COMPLEJIDAD TOTAL:  $O(n) + O(n) + O(m) + O(m \log m) =$ 
--                      (Radix)  (while) (list2Array) (mergeSort)
--                       $= O(n + m + m \log m) = O(n + m \log m)$ 

-----
-- SOLUCIÓN ALTERNATIVA CON TRIE:
-- Aprovechamos que las libretas están acotadas por 10 digitos,
-- por lo tanto sus operaciones son  $O(1)$ .
proc OrdenarXPromedioTrie(in array<(int,int)> A): array<(int,float)>{

    var notasPorEstudiante: dictTrie<int, <int,int>> = {}
    --                      <libreta, (sumaNotas, cantidad)>

    for i = 0; i < |A|; i++                                --0(n)
        var libreta: int := A[i][0]                        --0(1)
        var nota: int := A[i][1]                            --0(1)

        if notasPorEstudiante.esta(libreta)                --0(1)
            var par: <(int,int)> := notasPorEstudiante.obtener(libreta) --0(1)
            par[0] := par[0] + nota                          --0(1)
            par[1] := par[1] + 1                             --0(1)
        else
            notasPorEstudiante.definir(libreta, tupla(nota, 1)) --0(1)
        endif
    endfor

    var res: array<(int,float)> := []
    var i: int := 0
    for each libreta in notasPorEstudiante                  --0(m)
        var sumaNotas: int := notasPorEstudiante.obtener(libreta)[0] --0(1)
        var cantidadNotas: int := notasPorEstudiante.obtener(libreta)[1] --0(1)
        var promedio: float := sumaNotas / cantidadNotas      --0(1)
        res[i] := tupla(libreta, promedio)                    --0(1)
        i++
    endfor

    mergeSort(res)      --  $O(m \log m)$ 
    return res
}
-- COMPLEJIDAD TOTAL:  $O(n) + O(m) + O(m \log m) = O(n + m \log m)$ 
--                      (for)  (for each)  (mergeSort)

```